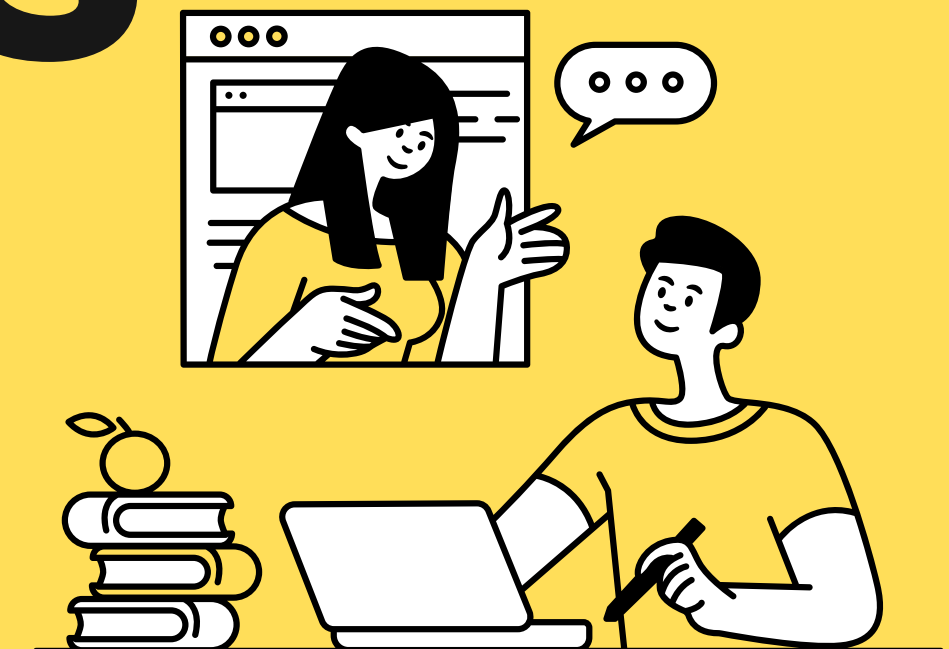




STUDENT ENGAGEMENT & PERFORMANCE ANALYSIS

USING SQL

An end-to-end data analysis project
using Structured Query Language on
an Online Learning Platform



INTRODUCTION

This project is part of the 100 Queries SQL Challenge, focused on analyzing an Online Learning Platform to understand student behavior, engagement patterns, and learning performance.

Using Structured Query Language (SQL), the project explores real-world business questions such as student activity trends, course performance, enrollment behavior, and overall platform health.

The objective is to transform raw data into meaningful insights that support data-driven decision making in educational platforms.





ER-DIAGRAM & DATABASE STRUCTURE

DATASET OVERVIEW

1. STUDENTS

Contains student profile and signup information.

2. COURSES

Includes course details such as course name, category, and price.

3. ENROLLMENTS

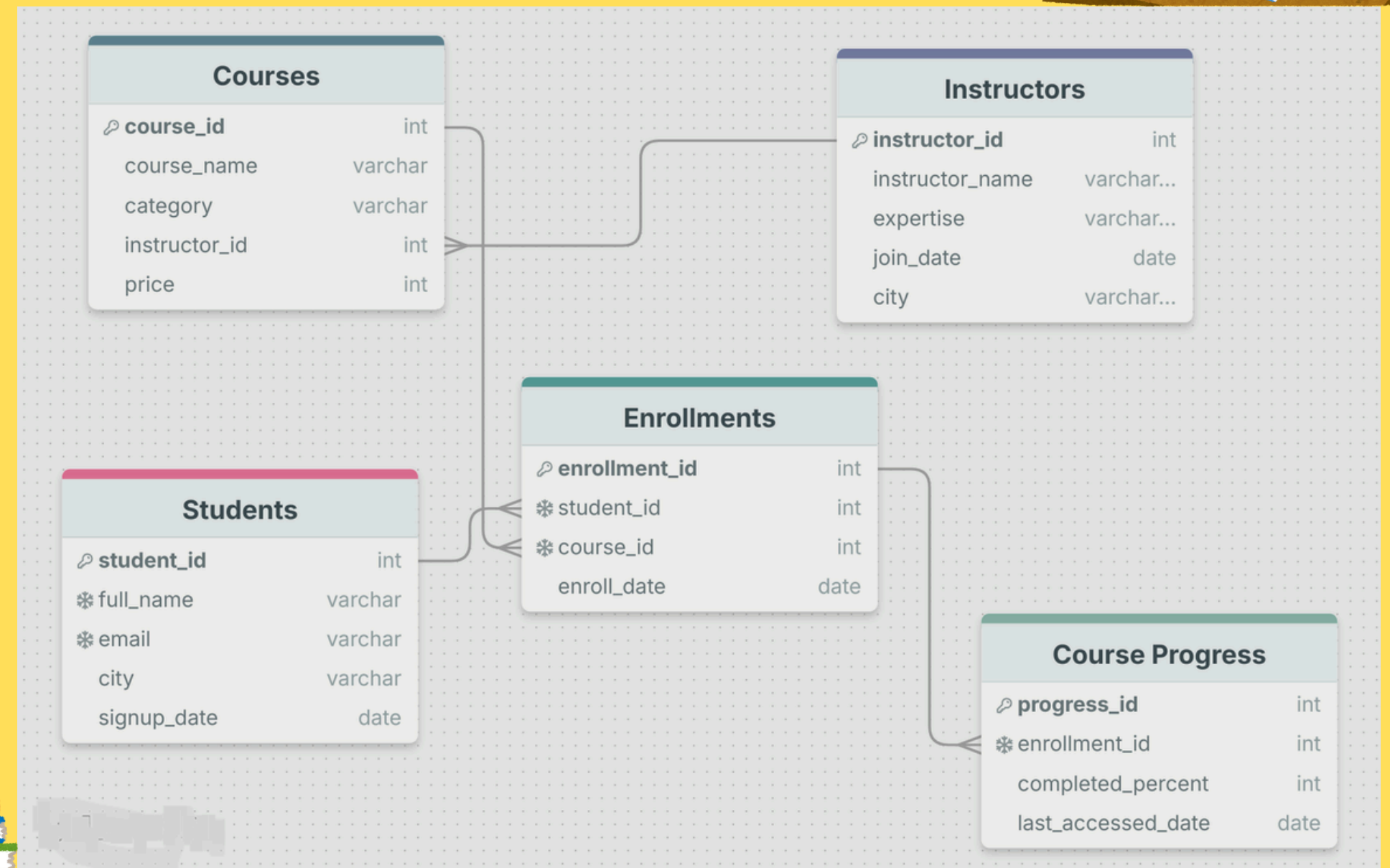
Tracks which students enrolled in which courses and when.

4. COURSE PROGRESS

Stores learning progress of each student for enrolled courses, including completion percentage

5. INSTRUCTORS

Stores instructor's data including enroll date,



1. PROJECT SETUP & BASIC QUERIES

In the first phase of the challenge, the primary focus was on understanding the dataset structure and establishing a strong foundation in SQL. This involved exploring the relationships between core tables such as Students, Courses, Enrollments, Course_Progress, and Instructors, and validating the data for accuracy and consistency. Basic SQL operations including SELECT, WHERE, ORDER BY, LIMIT, and simple JOIN statements were used to retrieve meaningful information.



Some of those Queries are:

```
-- 7.top10 most expensive courses(higher first)
select course_name,category,price from courses
order by price desc
limit 10;
```

```
-- 9.fetch courses whose category containing "data"-
select course_name,category from courses
where category regexp "data";
```

```
-- 17.show students who signed up after 2023-01-01--
select student_id,full_name,signup_date from students
where signup_date>'2023-01-01'
order by signup_date asc;
```

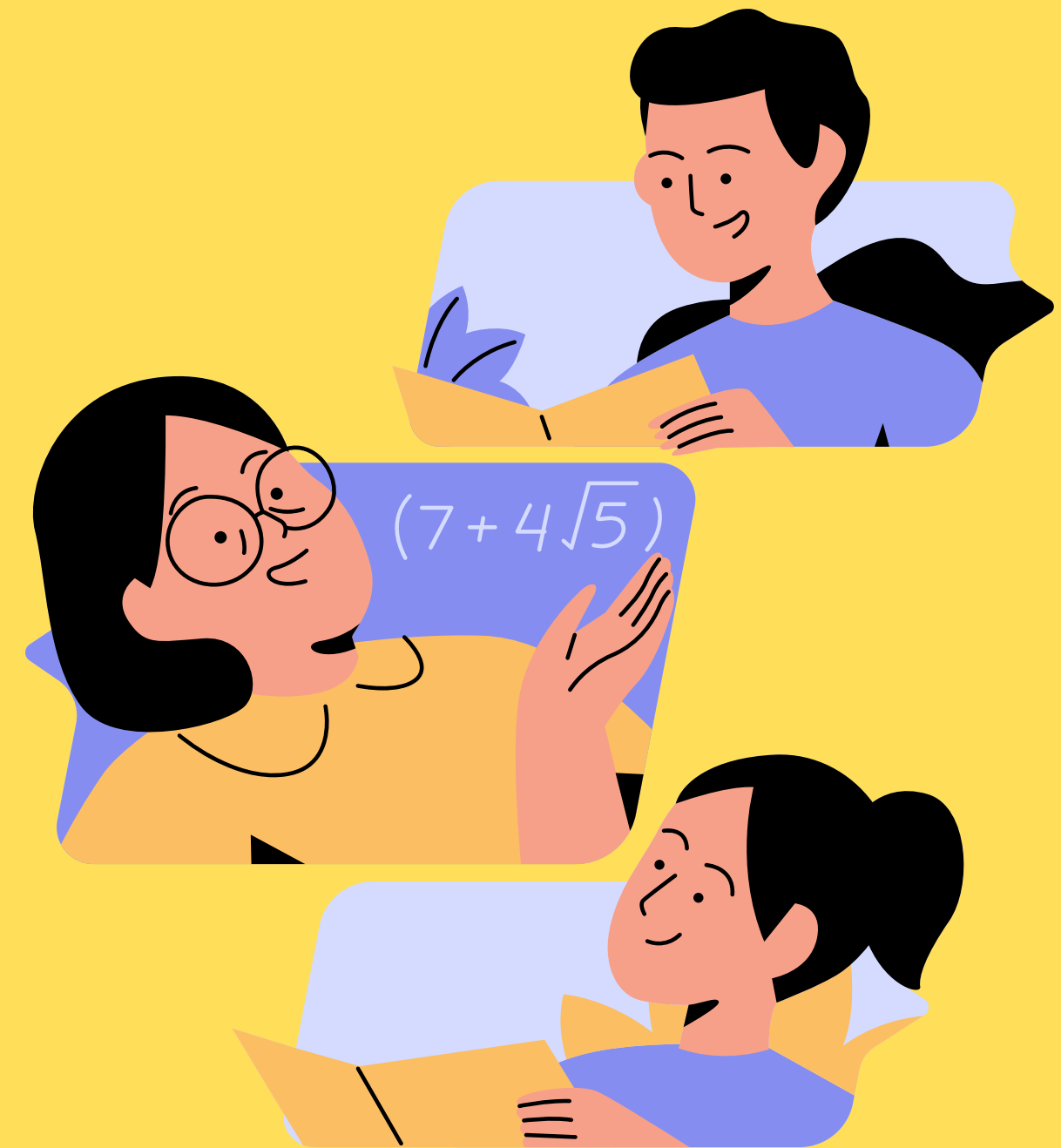
```
-- 11. count total num of courses in each category --
select distinct category,count(*)
as num_of_courses from courses group by category
order by num_of_courses desc;
```

```
-- 15.display 5 cheap courses(price in asc)
select course_id,course_name,category,price
from courses order by price asc limit 5;
```



2.CORE SQL & AGGREGATION

In the second phase, the focus shifted toward strengthening core SQL concepts related to aggregation and multi-table analysis. This stage involved extensive use of GROUP BY, HAVING, and aggregate functions such as COUNT, SUM, AVG, MIN, and MAX to generate meaningful summaries from the data. Key tasks included analyzing enrollment distributions, calculating average course completion rates, identifying highly active students, and measuring overall platform engagement. This phase helped transform raw data into structured insights and improved the ability to write efficient analytical queries.



Some of those Queries are:

```
-- 18.top 3 students who spent highest total amount--
select s.full_name as student_name,sum(c.price) as amount_spent
from students s join enrollments e on s.student_id=e.student_id
join courses c on e.course_id=c.course_id group by s.full_name
order by amount_spent desc limit 3;
```

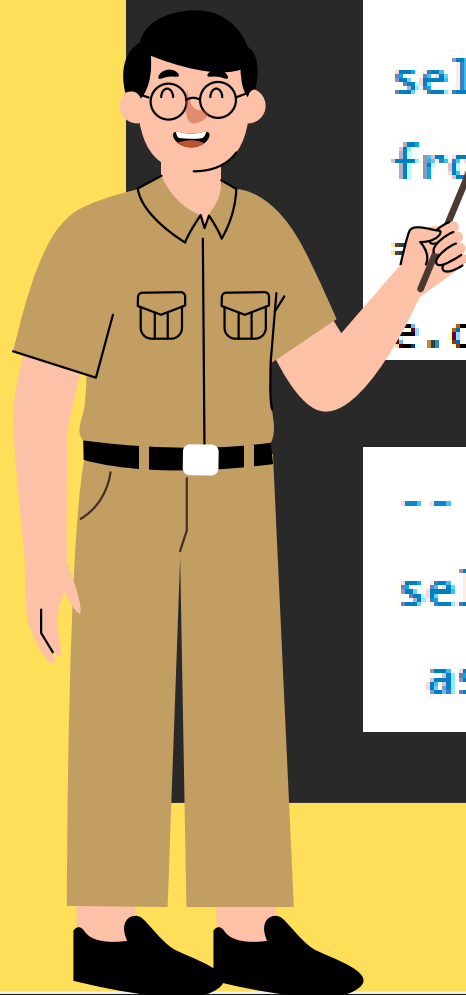
```
-- 7.total amount spent by each student--
select s.full_name,sum(c.price) as amount_spent
from students s join enrollments e on s.student_id
=e.student_id join courses c on
e.course_id=c.course_id group by s.full_name;
```

```
-- 9.avg course price category_wise--
select distinct category ,round(avg(price),2)
as avg_price from courses group by category;
```

```
-- 11.display courses with zero enrollments
select c.course_name,count(e.enrollment_id)
enrollments from courses c left join enrollments e
on c.course_id=e.course_id group by c.course_name
having enrollments =0;
```

```
-- 4.total enrollments per course--
select c.course_name,count(e.enrollment_id)
as num_of_enrollments from courses c join enrollments
e on c.course_id=e.course_id group by c.course_name;
```

```
-- 17.show students who have enrolled but have 0 progress--
select s.full_name,e.enrollment_id,c.completed_percent from
students s join enrollments e on s.student_id=e.student_id
join course_progress c on e.enrollment_id=c.enrollment_id
where c.completed_percent=0;
```



3.ANALYTICAL THINKING



The third phase emphasized analytical reasoning and problem-solving by converting business-style questions into SQL solutions. During this stage, complex scenarios such as identifying declining student engagement, evaluating learning trends over time, comparing performance metrics, and understanding behavioral patterns were addressed. Window functions, subqueries, and time-based analysis were used to explore trends and correlations. This phase significantly enhanced the ability to extract deeper insights and perform real-world analytical thinking using SQL.

Some of those Queries are:

```
-- 8. students enrolled in atleast 2 different categories--
select s.full_name,count(c.category) as categories_enrolled
from students s join enrollments e on s.student_id=
e.student_id join courses c on e.course_id=c.course_id
group by s.full_name having count(c.category)>2;
```

```
-- 19.display courses with enrollments greater than median--
select c.course_id,c.course_name,count(e.enrollment_id) as
enrollments from courses c join enrollments e on c.course_id=
e.course_id group by c.course_id,c.course_name having count
(e.enrollment_id)>(select avg(enrollments) from (select
count(enrollment_id) as enrollments from enrollments group by
course_id) t);
```

```
-- 18.find students who enrolled in second most expensive course--
select s.student_id,s.full_name,c.price as course_price from students s
join enrollments e on s.student_id=e.student_id join courses c on
e.course_id=c.course_id where c.price in ( select price from(select
price,dense_rank() over (order by price desc) as rnk from courses)t
where rnk=2);
```

```
-- 4.show students who completed atleast one course fully--
select s.full_name,count(e.enrollment_id) as enrolls,cp.completed_percent
from students s join enrollments e on s.student_id=e.student_id
join course_progress cp on e.enrollment_id=cp.enrollment_id where
completed_percent='100'group by s.full_name order by enrolls desc;
```



4. BUSINESS LOGIC & CONDITIONAL ANALYSIS



In the fourth phase, business-oriented analysis was introduced by applying conditional logic and advanced filtering techniques. This stage involved using CASE expressions, nested conditions, EXISTS, and complex joins to answer strategic questions such as identifying high-value students, determining courses that drive new user acquisition, and detecting courses with high enrollment but low completion rates. This phase bridged the gap between technical SQL skills and real business decision-making.

Some of those Queries are:

```
-- 15.find busiest enrollment date
SELECT enroll_date, COUNT(enrollment_id)
AS enrolls FROM enrollments GROUP BY
enroll_date HAVING COUNT(enrollment_id)
= (SELECT MAX(daily_count) FROM (
SELECT COUNT(enrollment_id) AS daily_count
FROM enrollments GROUP BY enroll_date) t);
```

```
-- 4.find total_enrollments yearwise
select year(enroll_date) as Year,count
(enrollment_id) as num_of_enrolls
from enrollments group by year(enroll_date);
```

```
-- 16.rank students based on total course value--
select s.student_id,s.full_name as high_value_students,rank()
over(order by sum(c.price) desc) as ranking from students s
join enrollments e on s.student_id=e.student_id join
courses c on e.course_id=c.course_id group by s.student_id,
s.full_name order by ranking asc;
```

```
-- 2.label students as old or new based on signup_date
select student_id,full_name,case
when year(signup_date)>=2024 then "New"
when year(signup_date)<2024 then "Old"
else "NAN"
end as student_catgory,signup_date from students;
```



5.ADVANCED SQL & REAL-WORLD ANALYTICS

The final phase focused on advanced SQL techniques and real-world analytics use cases. This included working with Common Table Expressions (CTEs), window functions, ranking methods, cumulative calculations, and performance-based segmentation. Advanced analyses such as platform-level health evaluation, Pareto analysis for revenue contribution, and overall system performance assessment were conducted. This phase demonstrated the ability to handle complex analytical challenges and deliver comprehensive insights suitable for professional data analysis environments.



Some of those Queries are:

```
-- 1. total course purchase value for each student--  
with value as(select s.student_id,s.full_name,sum(c.price)  
as course_purchase_value from students s join enrollments  
e on s.student_id=e.student_id join courses c on  
e.course_id=c.course_id group by s.student_id,s.full_name)  
select student_id,full_name,course_purchase_value from value;
```

```
-- 4.identify top10 high value students based on course spending  
select s.student_id,s.full_name as high_value_students,  
sum(c.price) as course_spending from students s join  
enrollments e on s.student_id=e.student_id join courses c on  
e.course_id=c.course_id group by s.student_id,s.full_name  
order by course spending desc limit 10;
```

```
-- 10.month wise enrollment count and compare it with previous month  
select distinct month(enroll_date) as month, count(*) as course_enrolls  
,lag(count(*)) over (order by month(enroll_date)) as Previous_month_enrolls  
from enrollments group by month(enroll_date) order by month;
```

```
-- 3.rank students based on total course spending from high to low  
select s.student_id,s.full_name ,sum(c.price) as course_spending,  
rank() over(order by sum(c.price) desc) as ranking  
from students s join enrollments e on s.student_id=e.student_id  
join courses c on e.course_id=c.course_id group by s.student_id,  
s.full_name order by ranking asc;
```





CONCLUSION

This project successfully demonstrates the practical application of SQL in analyzing an online learning platform through a structured 100 Queries SQL Challenge. By progressing from foundational querying to advanced analytical techniques, the project delivers meaningful insights into student engagement, learning behavior, course performance, and overall platform health. The use of real-world datasets, combined with analytical reasoning and business-focused problem solving, highlights the ability to convert raw data into actionable intelligence. This challenge not only strengthened my technical SQL proficiency but also enhanced critical thinking and data-driven decision-making skills, making the project a valuable demonstration of end-to-end data analysis capability.



THANK YOU!



Author

KOTARU VENKATA SAHITH

Email

SAHITHKOTARU@GMAIL.COM

Contact at

7382296484